Data Splitting and Validation

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why training data and testing data must be kept separate, and what generalization means.
- Distinguish overfitting from under-fitting using training-versus-testing accuracy gaps.
- Build a train-test split and a train-validation-test split in code using train_test_split.
- Apply random_state to make a split reproducible, and pick a split ratio that fits a dataset's size.

2. Detailed Explanation

Why training and testing data must stay separate

A model needs two distinct kinds of data: one to learn from and one to check whether it actually learned anything. **Training** is the process of teaching the model to learn the underlying patterns in the data. **Validation/testing** is the process of checking whether the model has learned those patterns correctly.

Think about a teacher who gives students a set of questions in class and then uses the exact same questions in the test. Every student scores well regardless of whether they understood the material, so the test becomes useless. A model evaluated on the same data it trained on behaves the same way — it can look perfect while having learned nothing. Call this a "pseudo-knowledgeable" model: it creates a false impression of competence and fails on new, unseen data.

Two more ways to picture this:

- **The chef test:** if a chef prepares the same dish 100 times and is tested only on that dish, the test proves nothing about general cooking skill. Testing on a different dish is the real test — just as a model must be tested on data different from what it trained on.
- **Cricket net practice vs. the real match:** training data is like net practice, where a player can request the deliveries they want and repeat specific shots. Testing data is like the real match, where the bowler reveals nothing in advance and the player faces genuinely unknown patterns. The real match is the true test of skill.

A worked example makes this concrete. Take a small dataset of hours studied (1 through 8) mapped to marks scored, where the relationship works out to:

marks = hours × 10 + 10
Asking the model to predict marks for an hours value it already trained on (say, 5) is a weak test — a correct answer could just mean the model memorized that value. Asking it to predict marks

for an hours value it never saw during training (say, 6) is a much stronger test. A correct prediction there is real evidence the model learned the underlying relationship rather than memorizing answers.

Generalization: the real goal of machine learning

Generalization means a model should perform well on unseen data, not just on the data it trained on. This is the most important concept and the backbone of the entire discipline of machine learning — "unseen data" here means the testing or validation data. The ultimate aim of building any model is to generalize properly: to perform well on data it has never encountered before.

Two analogies capture the idea:

- **Teach someone to fish:** "Don't give me a fish, teach me how to catch a fish." A ready-made fish only helps for a day; learning how to catch fish creates lasting, independent capability. A generalizing model learns the underlying pattern instead of memorizing a fixed set of correct answers.
- **The multiplication table:** suppose a student is taught 2×2=4, 2×3=6, and 2×4=8. If the student can then correctly work out 2×6, 2×7, 2×8, and 2×9, that demonstrates genuine learning of the pattern rather than memorization of specific facts.

Overfitting and underfitting

Overfitting describes a model that performs very well on training data but poorly on testing data. It has forcefully memorized the training patterns instead of learning the general relationship — it "hard-codes" itself to the training data and cannot generalize to new data. A model with 99% training accuracy but only 60% test accuracy is overfitting. Compare that to a second model with 98% training accuracy and 95% test accuracy. Even though its training accuracy is slightly lower, it is the better model, because the gap between training and testing performance is small.

Under-fitting is when a model does not even perform well on the training data — it has not learned the relationships properly because it is too simple or has not been trained enough. A simple model is more prone to under-fitting, while a complex model with many parameters is more likely to memorize the data and overfit.

The model you actually want has strong **generalization power** — good performance on both training and testing data.

Watch out for: a high training accuracy alone tells you nothing about quality. Always check the gap between training and testing accuracy before deciding a model is good.

The train-test split in code

Suppose 100 questions are used both to teach and test a student, and questions are picked randomly *with replacement* — a picked question goes back into the pool and can be picked again. There's a high chance the same question shows up in both the teaching set and the test set, and that repetition undermines the validity of the test.

The fix is to split the data up front — for example, 80% of the questions reserved purely for teaching and 20% reserved purely for testing. Only then sample **without replacement** within each partition: once a row is picked, it can't be picked again. This is the core idea behind train-test splitting. Training data teaches the model the underlying patterns; testing data evaluates how well it learned them.

Building the sample dataset:

```
import pandas as pd
# Sample dataset: hours studied vs marks scored
# Marks follow the relationship marks = hours * 10 + 10
data = pd.DataFrame({
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8],
    "Marks": [20, 30, 40, 50, 60, 70, 80, 90]
})
```

Before any splitting happens, separate the input features (X) from the target variable (y):

```
X = data[["Hours"]]   # double brackets: keeps X as a DataFrame (list of features)
y = data["Marks"]     # target variable
```

The double square brackets are deliberate — since a real dataset can have multiple feature columns, X is written as a list of column names even when there's only one.

Now perform the split:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 42
)
```

A few details matter here:

- train_test_split must be imported from sklearn.model_selection, **not** sklearn.preprocessing — a common mistake.
- The order of the returned variables matters: X_train, X_test, y_train, y_test.

- test_size is a **fraction**, not a percentage — 0.2 means 20% of the data goes to testing (and 80% to training).
- 80/20 and 75/25 are both very standard choices; there's no single fixed rule for the split ratio.
- random_state=42 is a common value, but any integer works (other examples: 45, 35, 32).
- Splitting happens without replacement — once a row lands in one partition, it can't also appear in another.
- train_test_split keeps X and y synchronized automatically. If their ordering ever became mismatched, an input row would end up paired with the wrong target value, corrupting the dataset.

Watch out for: if a dataset has 500 rows and test_size=0.1, that means 10% (50 rows) goes to testing and 90% (450 rows) goes to training. Always read test_size as "share going to the *test* set," not the training set.

random_state **and reproducibility**

Every time a deck of playing cards (spades, hearts, diamonds, clubs) is shuffled, a different random order results. In exactly the same way, every time train_test_split runs **without** a random_state, it produces a different random split of the rows.

Running the split repeatedly without setting random_state produced different selected row indices each time — one run picked indices like 2,0,1,4,6,5, and the next run gave a completely different set. Running the split with random_state=42 produced the exact same split every time (training indices came out as 0,7,2,4,3,6 consistently). Changing the seed to random_state=50 produced a different but still fixed and reproducible split (indices such as 6,1,3,7,0), and re-running with seed 50 again reliably reproduced that same result.

This shows that random_state fixes the "shuffle" so the same split can be regenerated on demand — comparable to reusing an already-shuffled deck instead of reshuffling every time. Without it, results are not reproducible from run to run.

Validation sets and three-way splitting

Suppose five different models — jokingly nicknamed "Jarvis 1" through "Jarvis 5" — have been built for the same task, and the best one needs to be picked. If a model is evaluated repeatedly on the very same test set, that test set effectively stops being "unseen," because tuning choices start being indirectly influenced by it. This is the motivation for a third, separate portion of data — the **validation set** — sitting between training and final testing.

Industry practice, in contrast to the commonly-taught two-way split, uses **three** portions of data:

Portion	Purpose
Training data	The model learns the underlying patterns from this data.
Validation data	Used to tune hyperparameters.
Test data	Used for final evaluation of the model's performance.

Hyperparameter tuning is the process of finding the best settings or configuration for a model — the parameter values that make it perform best.

This mirrors preparing for board exams: a student studies, takes a mock test ("prelims" or pre-board) to gauge performance and see where to focus more effort, and only afterward sits the final exam. Study, then mock test, then final exam parallels train, then validate, then test.

Because train_test_split only produces two outputs at a time, three-way splitting is built by calling it twice.

- Step 1 — split off the training portion:

```
from sklearn.model_selection import train_test_split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size = 0.3, random_state = 42
)
```

This puts 70% of the rows into X_train/y_train and the remaining 30% into a temporary holding set, deliberately named X_temp/y_temp (not "test") because it still needs to be divided further.

- Step 2 — split the temporary portion into validation and test:

```
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
```

This splits the 30% held in X_temp/y_temp evenly in half, yielding 15% of the original data for validation and 15% for test.

Watch out for these common mistakes at this step:

1. The parameter is always named test_size, even when the result is a validation set — there's no separate temp_size parameter.
2. In the second split, the inputs must be X_temp and y_temp — **not** the original X and y. Passing the original X and y again is the most common error people make here.
3. Naming the outputs temp, val, or anything else is entirely up to the person writing the code — the function being called train_test_split doesn't mean the outputs have to be literally called "test."

The end-to-end machine learning pipeline

Data splitting is one stage inside a larger workflow. Here's the simplified, non-technical overview of the full pipeline:

A few notes on individual stages: the **problem statement** is the most fundamental starting point, even before collecting data, since it determines what data needs to be collected. **EDA (Exploratory Data Analysis)**, feature engineering, and feature scaling can be arranged flexibly relative to one another, since EDA is a fairly generic step. **Deployment** means making the finished model available to the outside world. For example, a house price prediction model could run on a website where a user enters square footage, number of BHKs (bedroom-hall-kitchen), and amenities, then clicks "Find Price" to get a prediction. This is what "productizing" a model means. **Monitoring** tracks the model's real-world performance after deployment, and that feedback loops back to improve the model further.

Standard split ratios and why they change with dataset size

There's no fixed, universal rule for split ratios — the right ratio is subjective and depends on company, business context, and dataset size. These are common, not mandatory, conventions:

Dataset size	**Split used**	**Notes**
Small dataset	90% train / 10% test	No validation split — validation is optional here.
Medium dataset	80% train / 20% test	Test share increases slightly.
Large dataset	70% train / 15% validation / 15% test	Validation is introduced once size allows it.

Very large dataset	60% train / 20% validation / 20% test	The percentage allocated to train decreases further.

Even though the training *percentage* decreases for larger datasets, the absolute number of training rows can still be very large. For example, 70% of 100 rows is only 70 rows, but 60% of 1,000 rows is 600 rows — a much larger absolute training set despite the lower percentage. That's why very large datasets can afford a smaller training share while still leaving plenty for validation and testing.

Validation is described as a **luxury** on top of the mandatory needs of training and testing:

- **One hour before an exam:** spend it on the necessity (studying) rather than the luxuries (watching a show, going out). A brief indulgence — a two-minute Maggi noodle break, then studying the rest of the time — is fine. With six hours instead of one, that extra time could reasonably be split between the necessity and the luxuries.
- **Rent vs. shopping:** with only ₹20,000 available and rent due, pay the rent (necessity) before anything else — shopping is the luxury. With ₹1,00,000 available and rent of only ₹30,000, there's enough to cover both the necessity and some of the luxury.

The underlying point: mandatory needs (training and testing) must be satisfied first; validation, as a luxury, gets added once the dataset is large enough to comfortably support it.

3. Key Takeaways

- Training data teaches a model; testing and validation data check whether it actually learned — never grade a model on the data it studied from.
- Generalization — performing well on unseen data — is the central goal of machine learning; overfitting (great on training, poor on testing) and underfitting (poor even on training) are both failures to generalize.
- train_test_split from sklearn.model_selection takes X, y, test_size (a fraction), and random_state (an integer seed), and returns X_train, X_test, y_train, y_test in that order.
- random_state fixes the randomness of a split so it can be exactly reproduced; without it, every run produces a different split.
- A three-way train/validation/test split — built with two successive calls to train_test_split — adds a validation set for hyperparameter tuning that stays independent of the final test set.
- Standard split ratios (90/10, 80/20, 70/15/15, 60/20/20) shift with dataset size, because larger datasets sustain smaller training percentages while still producing large absolute training sets.

Mental model: think of a dataset like exam preparation. Training is studying, validation is the mock test that tells you where to improve, and the final test is the real exam that no amount of memorization can fake.